

EXP 8 DEADLOCK AVOIDANCE (Banker's Algorithm)

AIM:

To find out a safe sequence using Banker's Algorithm for deadlock avoidance.

ALGORITHM:

1. Initialize work = available and finish[i] = false for all processes.
2. Find an index i such that:
 - finish[i] == false
 - need[i] ≤ work
3. If no such i exists, go to Step 6.
4. Update:
 - work = work + allocation[i]
5. Set finish[i] = true and repeat from Step 2.
6. If all finish[i] == true, then a safe sequence exists.
7. Otherwise, no safe sequence exists (deadlock condition).

CODE:

```
#include <stdio.h>

int main()
{
    // Number of processes and resources
    int n = 5;
    int m = 3;
    int i, j, k;

    // Allocation Matrix
    int alloc[5][3] = {
        {0, 1, 0}, // P0
        {2, 0, 0}, // P1
        {3, 0, 2}, // P2
        {2, 1, 1}, // P3
        {0, 0, 2}  // P4
    };
};
```

```
// Maximum Matrix
```

```
int max[5][3] = {  
    {7, 5, 3}, // P0  
    {3, 2, 2}, // P1  
    {9, 0, 2}, // P2  
    {2, 2, 2}, // P3  
    {4, 3, 3} // P4  
};
```

```
// Available Resources
```

```
int avail[3] = {3, 3, 2};
```

```
int f[5] = {0}; // Finish array
```

```
int ans[5]; // Safe sequence
```

```
int ind = 0;
```

```
// Need Matrix
```

```
int need[5][3];
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    for (j = 0; j < m; j++)
```

```
    {
```

```
        need[i][j] = max[i][j] - alloc[i][j];
```

```
    }
```

```
}
```

```
// Banker's Algorithm
```

```
for (k = 0; k < n; k++)
```

```
{
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        if (f[i] == 0)
```

```
        {
```

```
            int flag = 0;
```

```
            for (j = 0; j < m; j++)
```

```
            {
```

```

        if (need[i][j] > avail[j])
        {
            flag = 1;
            break;
        }
    }

    if (flag == 0)
    {
        ans[ind++] = i;

        for (j = 0; j < m; j++)
        {
            avail[j] += alloc[i][j];
        }

        f[i] = 1;
    }
}

}

}

printf("The SAFE Sequence is:\n");

for (i = 0; i < n - 1; i++)
    printf("P%d -> ", ans[i]);

printf("P%d\n", ans[n - 1]);

return 0;
}

```

RESULT:

Thus, the safe sequence was successfully determined using Banker's Algorithm, ensuring deadlock avoidance.